

HT:

T: Tkey []

next: Integer[]

mu: Integer

firstEmpty: Integer

h: TFunction

COALESCED CHAINING

remove (11)

elems	3	X		8	
next	-1	-1	-1	0	-1

mark pos. as empty

remove (11)

elems	X	56	12	8	
next	-1	0	-1	-1	-1

remove elem. from end of a ll on array ?

(11)

elems	20	11	56		
next	-1	2	-1	-1	-1

?

Open address. (probe seq.)

I. Linear probing: $h(k, i) = (h'(k) + i) \text{ mod } m$ $m=16$
 $h'(k) = k \% m.$

key	76	12	109	43	22	18	55	81	91	27	13	16	39
hash	12												

$$h'(76) = 76 \% 16 = 12$$

$$h(k, 0) = ((12 + 0) \% 16) = 12.$$

$$\begin{array}{r} 16 \\ \times 5 \\ \hline 80 \\ 16 \\ \times 6 \\ \hline 96 \\ 16 \\ \times 12 \\ \hline 192 \end{array}$$

- ⊖ primary clustering odd free
- ⊖ only $m \neq$ probe seq. even occupied.
- ⊕ probe seq. = permutation

II. Quadratic probing: $h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \% m$

III. Double hashing: $h(k, i) = (h'(k) + i \cdot h''(k)) \% m$

HT:

T: Tkey[]

m: Integer

h: Tfunction

OPEN ADDR.

$$\alpha = \frac{n}{m}$$

$$\frac{1}{1-\alpha} \quad \text{failed search.}$$

$$\frac{1}{\alpha} - \ln \frac{1}{1-\alpha} \quad \text{successful search.}$$

$$\alpha \text{ ct. } \Rightarrow \Theta(1)$$

$$wc: \Theta(m)$$

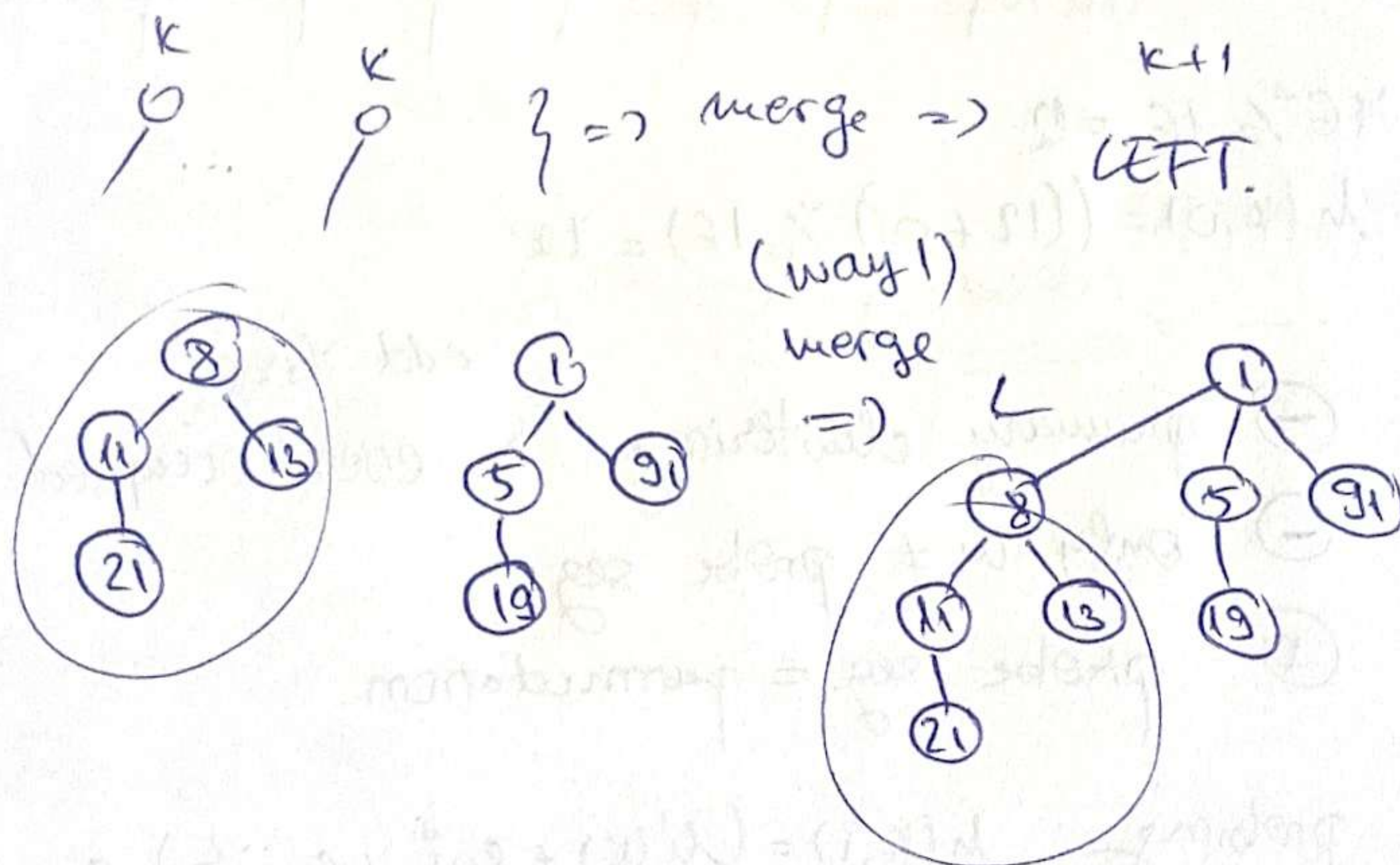
Lecture 9

Binomial heap = coll. of Binomial trees.

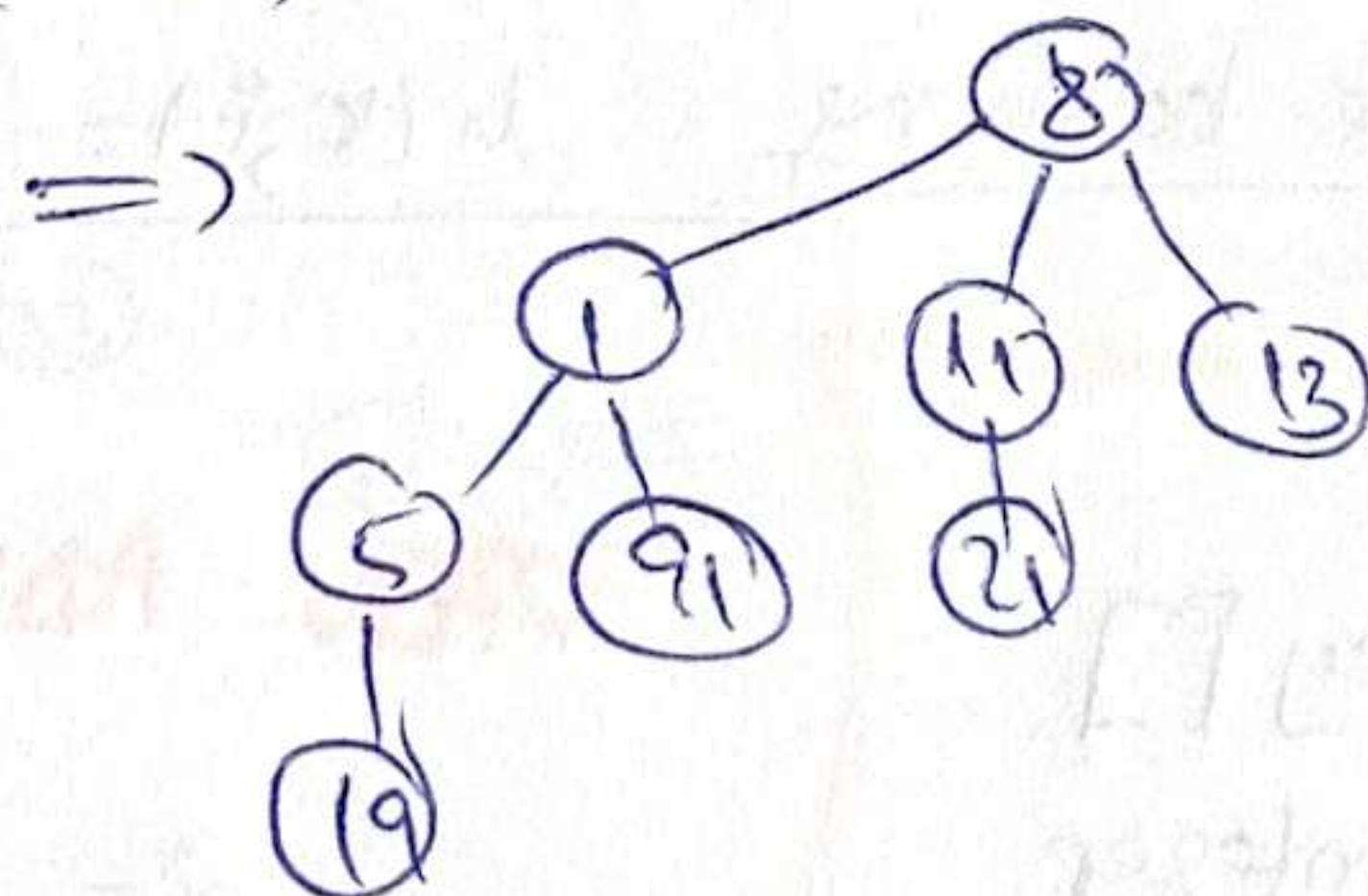
order $k \Rightarrow 2^k$ nodes.

height = k

delete root $\Rightarrow k$ binomial trees.



(way 2)



Binomial tree: - heap property

- Sorted LL $\rightarrow$ node = Binomial tree

$\hookrightarrow$ relation: order of the trees.

$14 = 1110 = 2^3 + 2^2 + 2^1 \Rightarrow$ Binomial heap with 14 nodes contains binomial trees of orders 3, 2, 1 (sorted: 1, 2, 3). $\Rightarrow \exists 3$ trees.

$21 = 10101 = 2^4 + 2^2 + 2^0 \Rightarrow$ orders: 4, 2, 0 $\Rightarrow \exists 3$ trees

- n elems $\Rightarrow$ max $\log_2 n$ bin. trees.

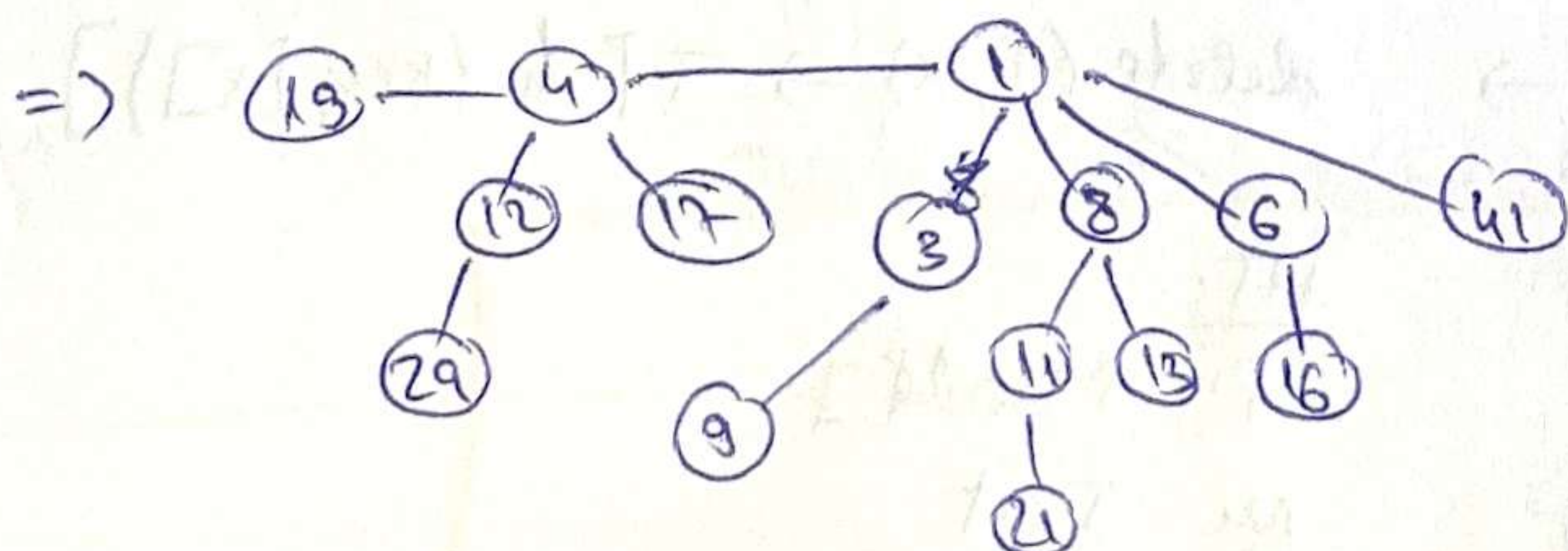
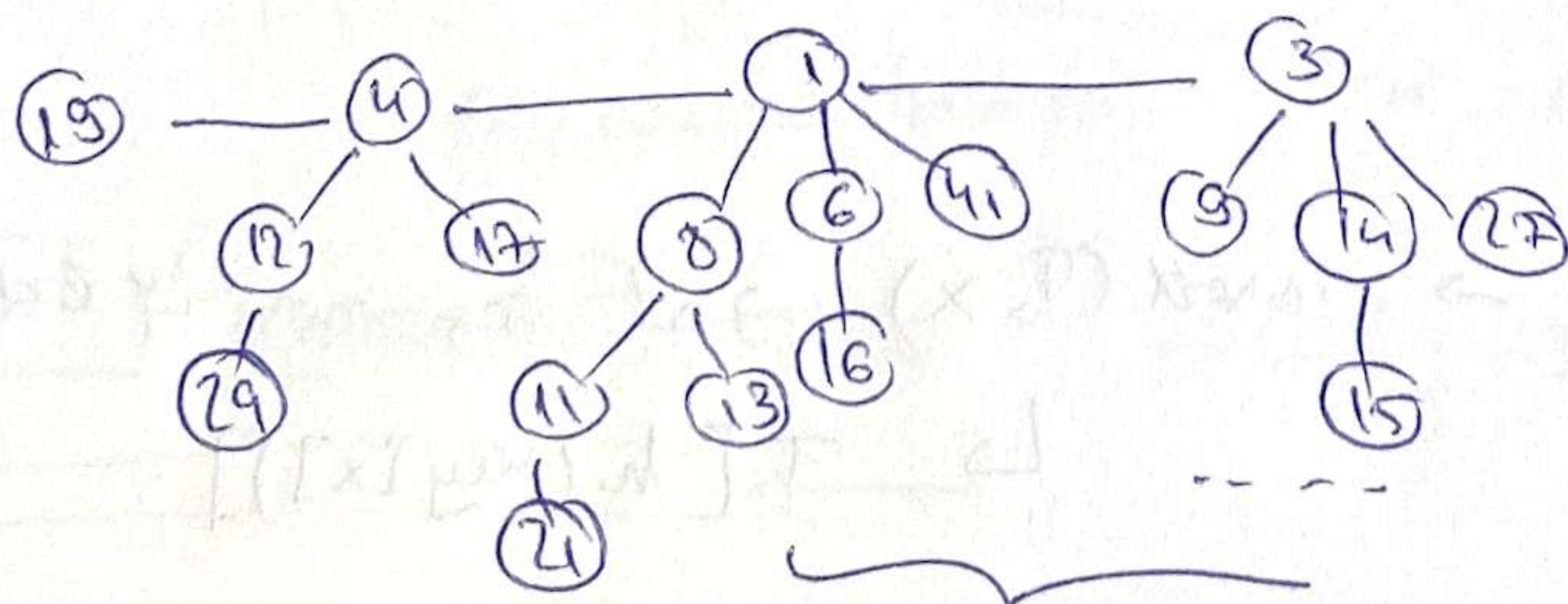
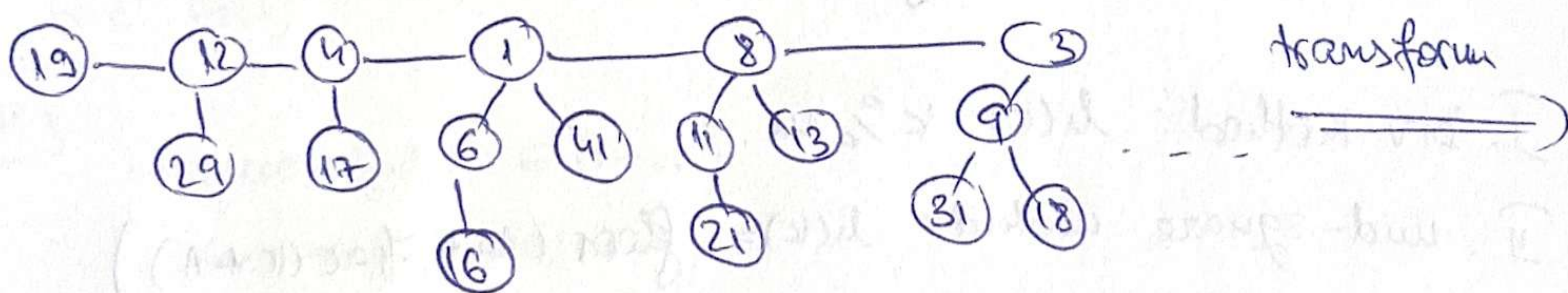
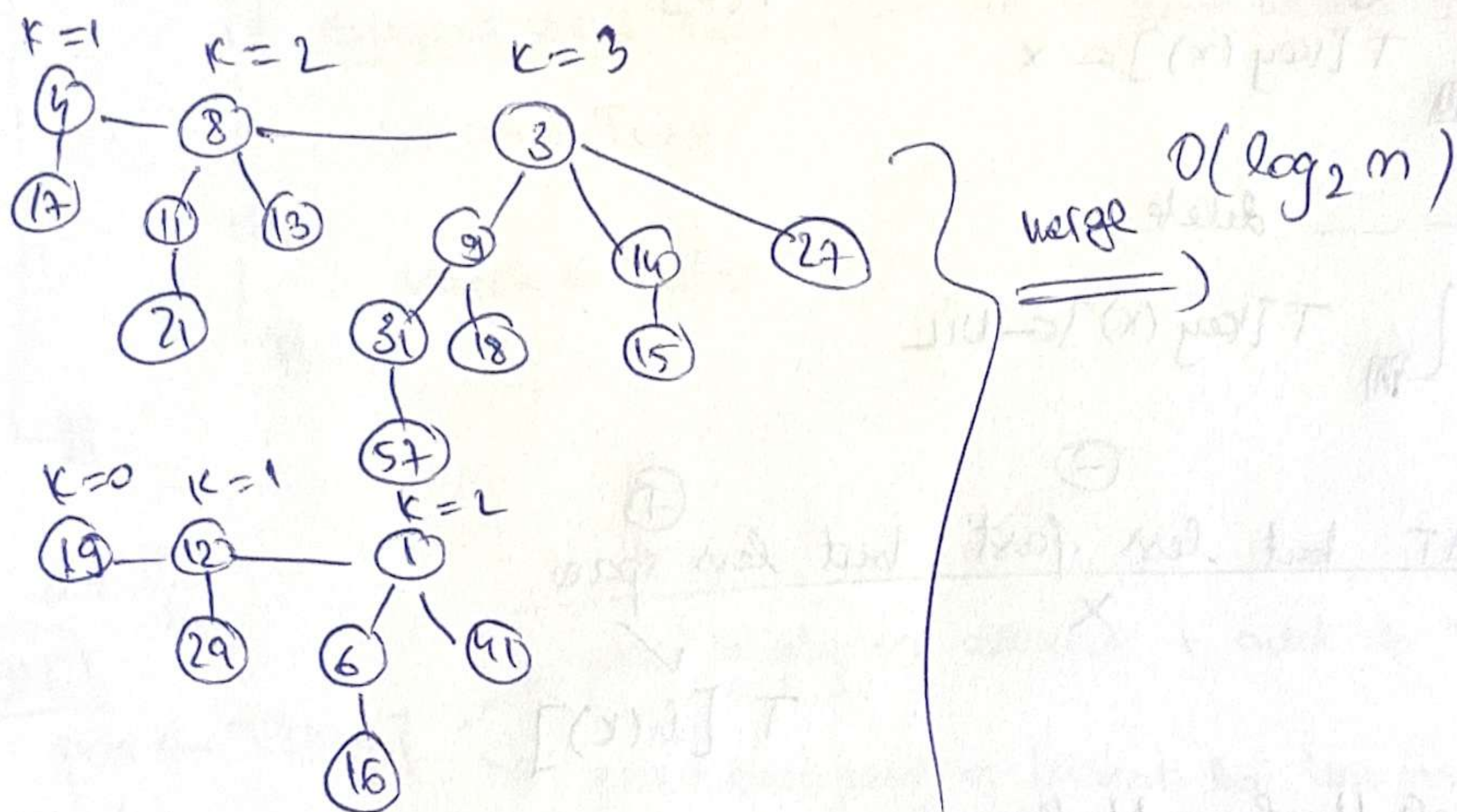
- $\Rightarrow$ ~~height~~ max

max $\log_2 n$ height.

[merge] $\exists$ 1st BH $\Rightarrow$ LL sorted
 $\exists$ 2nd BH $\Rightarrow$ LL sorted $\Rightarrow$ merge $\Rightarrow$ 3rd BH
~~1st, 2nd~~

Merge 2 sorted LL - heap property

- order $k \Rightarrow$ order $k+1$.



[push] $\rightarrow$ insert new elem $\rightarrow$ BH create $\oplus$ merge $\rightarrow O(\log_2 n)$

[top] $\rightarrow$ min. elem = root of one BT $\rightarrow$ check every root $\Rightarrow O(\log_2 n)$

pop $\rightarrow$ remove min $\rightarrow$ remove root

DAT function search(T, k) is:

$\hookrightarrow$ search $\in T[k]$

Subalg insert(T, x) is

$\hookrightarrow T[\text{key}(x)] \leftarrow x$

delete

$\hookrightarrow T[\text{key}(x)] \leftarrow \text{Nil}$

HT $\stackrel{\ominus}{=}$ DAT but less fast, but less space $\stackrel{\oplus}{\checkmark}$
 $T[h(k)]$

Simple Uniform Hashing

I. Div. Method: $h(k) = k \% m$.

II. mid-square method: $h(k) = \text{floor}(m + \text{frac}(k + A))$

III. univ. hashing

sep. chaining $\rightarrow$ insert(T, x) $\rightarrow$ at beginning of list.
 $\hookrightarrow T[h(\text{key}[x])]$

$\rightarrow$ search(T, k) $\rightarrow T[h(k)]$

$\rightarrow$ delete(T, x) $\rightarrow T[h(\text{key}[x])]$

Node:

~~info~~ key: Tkey

next: TNode

HT:

T: TNode[]

m: Int

h: TFunction

function search (ht, k) is:

```

pos ← ht.h(k)
current ← ht.T[pos]
while current ≠ Nil and [current].Key ≠ k ex
    current ← [current].next
    ↗
if current ≠ Nil then ← we stopped because of ⇒ found
    search ← True
else
    search ← False
    ↘

```

$$\alpha = \frac{m}{m}$$

INSERT

pos ← T[key] → pos = empty ⇒ allocate + add to slot.
 → pos = occupied ⇒ insert to beginning of list.

we: $\Theta(1)$

SEARCH

unsuccessful: $\Theta(1 + \alpha)$

successful: $\Theta(1 + \alpha)$

$\Theta(1)$ compute hash fct.

$\alpha = A.C$ to search one of the m lists.

Lectures

①

Queue

(ADT)

FIFO

→ **DS**: - DA (circular array)

- SL

- DL

front

↑

pop

rear

↑

push

↖ good if we push more (SL)

↑

$\Theta(n)$ both

↙

↖ good if we pop more (DL)

rear

front

⊕ Keep head, tail (efficient)

7 rev. node needed for removal.

DLL $\leftrightarrow$ does not matter where you put front (rear).

② Deque - insert/delete from both ends

ADT $\rightarrow$ STACKS, QUEUES

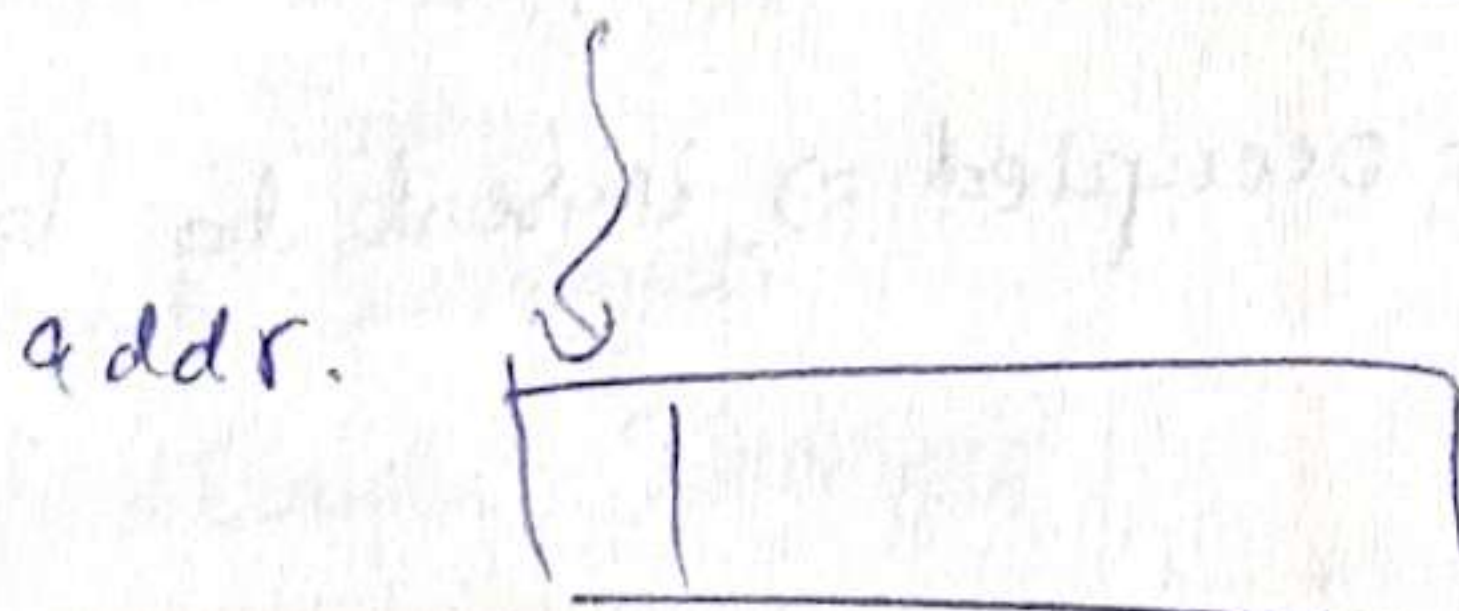
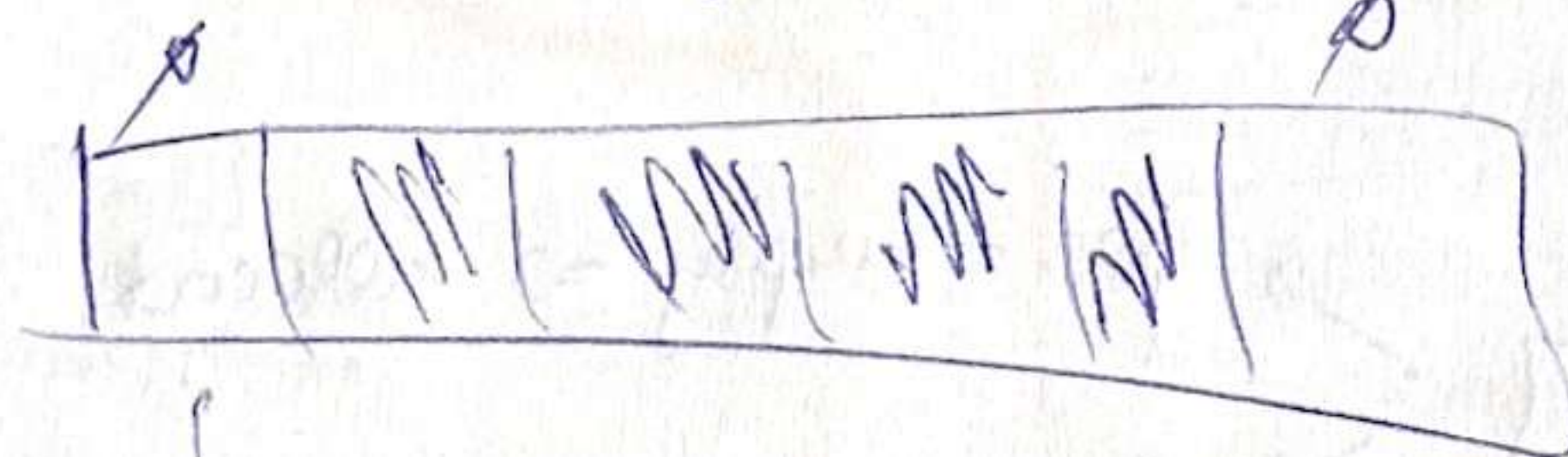
$\downarrow$

DS: - circular array

- DLL

- dyn. array of d.-size arrays

$\downarrow$



③ Priority Queue $\rightarrow$

ADT

DS:

- DA

- LL

- Bin. heap (DA + Bin. tree)

order by priority?

insertion order?

add $\rightarrow$ last of array $\downarrow$
 $\rightarrow$ bubble up $\rightarrow$

remove (root only) $\rightarrow$ last to the root $\rightarrow$ bubble-down.

Heap:

cap: Int

len: Int

elems: T elem

$\pm$ rel

Lecture 6

1) SLL on array

LL → elems stored in an array.

each elem → link. (not pointer → but position)

	0	1	2	3	4	5	6	7	8	9
elems		78	11	6	59	19		44		
next	7			-1						

head = 3

first Empty = 1

SLLA

elems: TElem[]
 next: Integer[] ← pos
 cap: Integer
 head: Integer
 firstEmpty: Integer

Array SLL

SLLAIterator

sll: SLLA
 current: Integer

2) DLL on array

DLLA Node

info: TElem
 next: Integer
 prev: Integer

DLLA

nodes: DLLANode[]
 cap: Integer
 head: Integer
 tail: Integer
 firstEmpty: Integer
 size: Integer

⊕ allocate ⊕ free

set head to 1

DLLAIterator

list dlla: DLLA
 current: Integer

~~it.dlla ←~~

it.list ← dlla
 it.current ← dlla.head

3) Stack (LIFO) $\rightarrow$ DS :- static array
 - dynamic array } arrays
 - SLL } LL
 - DLL

arrays : TOP at the end.

SLL : TOP at head

DLL : TOP at head/end.

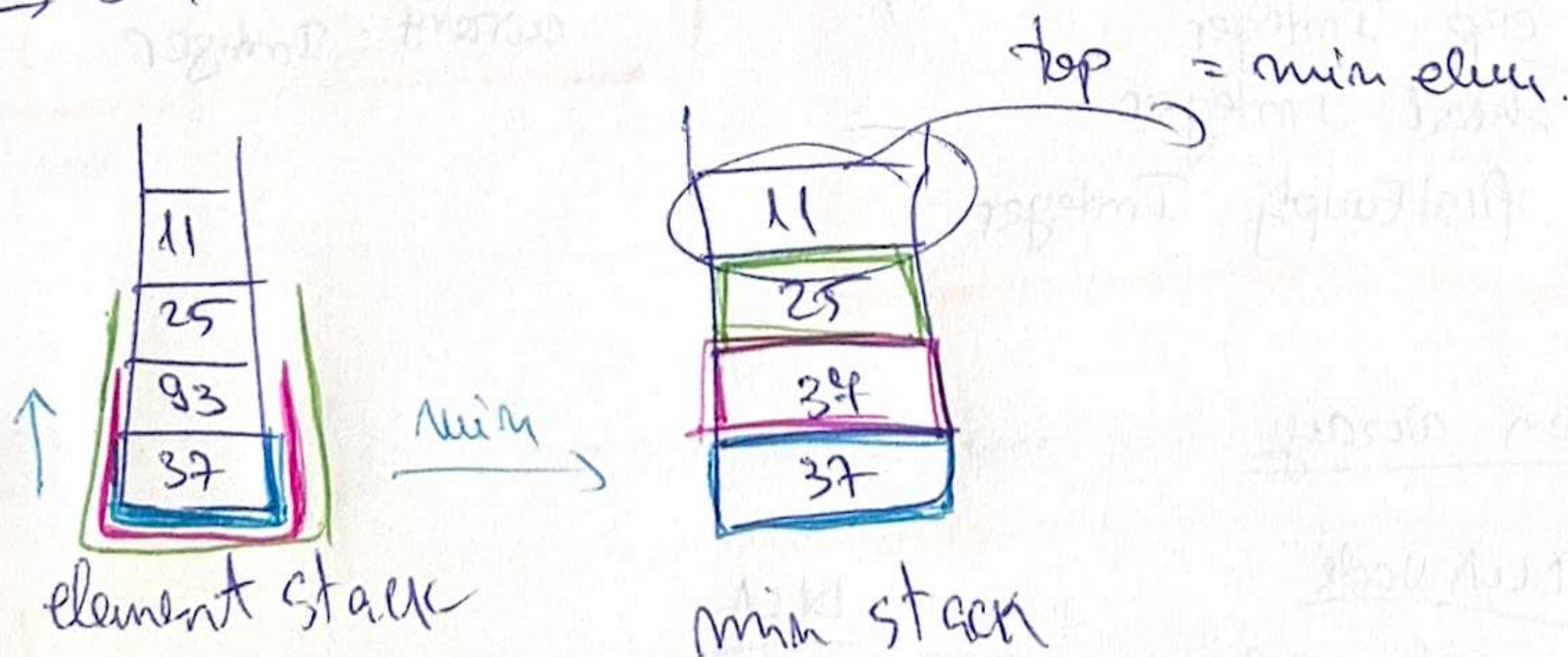
Stack w fixed max. cap using LL

⊕ maxCap (cap)

⊕ currentSize (nElements)

Special stack (min in $\Theta(1)$)

$\hookrightarrow$ aux stack



Lecture 6

1) XOR LL = DLL, every node has 1 link = (prev XOR next)

DLL > SLL.

⊕ eff.

⊖ memory

NIL XOR B = B

B: A XOR C



$$A \text{ XOR } C \text{ XOR } A = \underbrace{A \text{ XOR } A} \text{ XOR } C = \text{NIL XOR } C = C$$

XORNode

info: TElem

link: TInteger

XORList

head: TXORNode

tail: TXORNode

LL on arrays

	1	2	3	4	5	6
elems	46	78	11	6	59	19
next	5	6	1	-1	2	4

head = 3 $\Rightarrow$ first elem = 11 $\rightarrow$ next = 1 $\rightarrow$ 46 $\rightarrow$ next = 5 $\rightarrow$
 $\rightarrow$ next = 59 $\rightarrow$ next = 2 $\rightarrow$ 78 $\rightarrow$ next = 6 $\rightarrow$ 19
 $\rightarrow$ next = 4 $\rightarrow$ 6 $\rightarrow$ next = -1 stop.

$\Rightarrow$ 11, 46, 59, 78, 19, 6

remove 46 $\Rightarrow$ modify links $\Rightarrow$

11
5

$\Rightarrow$

elems	46	78	11	6	59	19
next		6	5	-1	2	4

insert 44 at 3rd pos $\rightarrow$ anywhere, but set links!

	1	2	3	4	5	6	7	8	9	10
elems		78	11	6	59	19		44		
next			5		8			2		

head

11, 59, ...

find empty pos $\Rightarrow O(n)$

$\Rightarrow$ need to keep a ll of empty pos.

firstEmpty = 1

SLL

elems: TElem[]

next: Integer[]

head: Integer

cap: Integer

firstEmpty: Integer

... Search (sll, elem) is

```
current ← sll.head
while current ≠ -1 and sll.elems[current] ≠ elem execute
    current ← sll.next[current]
if current ≠ -1 then // we stopped bc of
    search ← True
else
    search ← False
```

$O(n)$

insertPos

→ iterate until AFTER.

→ first pos (special case)

→ check valid pos.

→ array → check at every add if we still have space /
resize → also add in the list of empty pos.

lectures

1) SLL



SLLNode

info: TElem

next: ↑ SLLNode

SLL

head: ↑ SLLNode

(± fail)

SLLIterator

sll: SLL

currentElem: ↑ SLLNode

Subalg init(it, sll) is:

it.sll $\leftarrow$ sll

it.currentElement $\leftarrow$ sll.head

$\Theta(1)$

function getCurrent(it) is:

if it.currentElement = Nil then

@throw...

e $\leftarrow$ [it.currentElement].info

getCurrent $\leftarrow$ e

$\Theta(1)$

Subalg next(it) is:

if it.currentElement = Nil then

@throw...

it.currentElement $\leftarrow$ [it.currentElement].next

$\Theta(1)$

valid(it)...

if --- $\neq$ Nil

valid $\leftarrow$ True

$\Theta(1)$

2) DL

$\rightarrow$
 $\leftarrow$

DLNode

info: TElem

next: $\uparrow$ DLNode

prev: $\uparrow$ DLNode

DL

head: $\uparrow$ DLNode

tail: $\uparrow$ DLNode

$\leftarrow$ if str. of the list will be modif.

subalg insertLast (dll, elem) is:

newNode ← allocate()

[newNode].info ← elem

[newNode].next ← Nil

[newNode].prev ← dll.tail

if dll.head = Nil then // empty list

dll.head ← newNode

dll.tail ← newNode

else

[dll.tail].next ← newNode

dll.tail ← newNode

SU → stop AFTER
(insert)

DU → stop BEFORE/AFTER

subalg insertAtPos (dll, pos, elem) is:

if pos < 1 then

@ error

else if pos = 1 then

insertFirst (dll, elem)

else

currentNode ← dll.head

currentPos ← 1

while currentNode ≠ Nil and currentPos < pos - 1 ex

currentNode ← [currentNode].next

currentPos ← currentPos + 1

if currentNode = Nil then

@ -

else if currentNode = dll.tail then

insert (dll, elem)

else

newNode ← allocate()

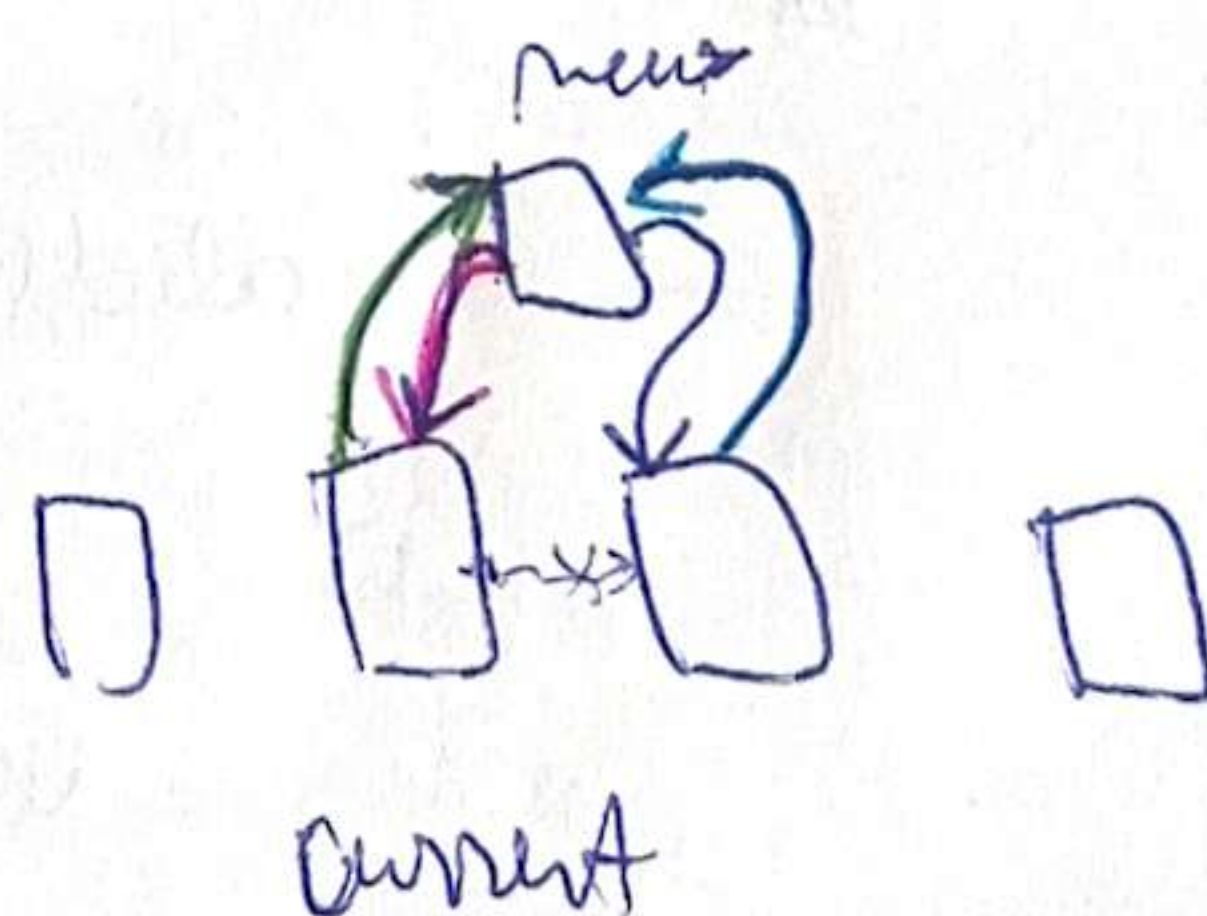
[newNode].info ← elem

[newNode].next ← [currentNode].next

[newNode].prev ← currentNode

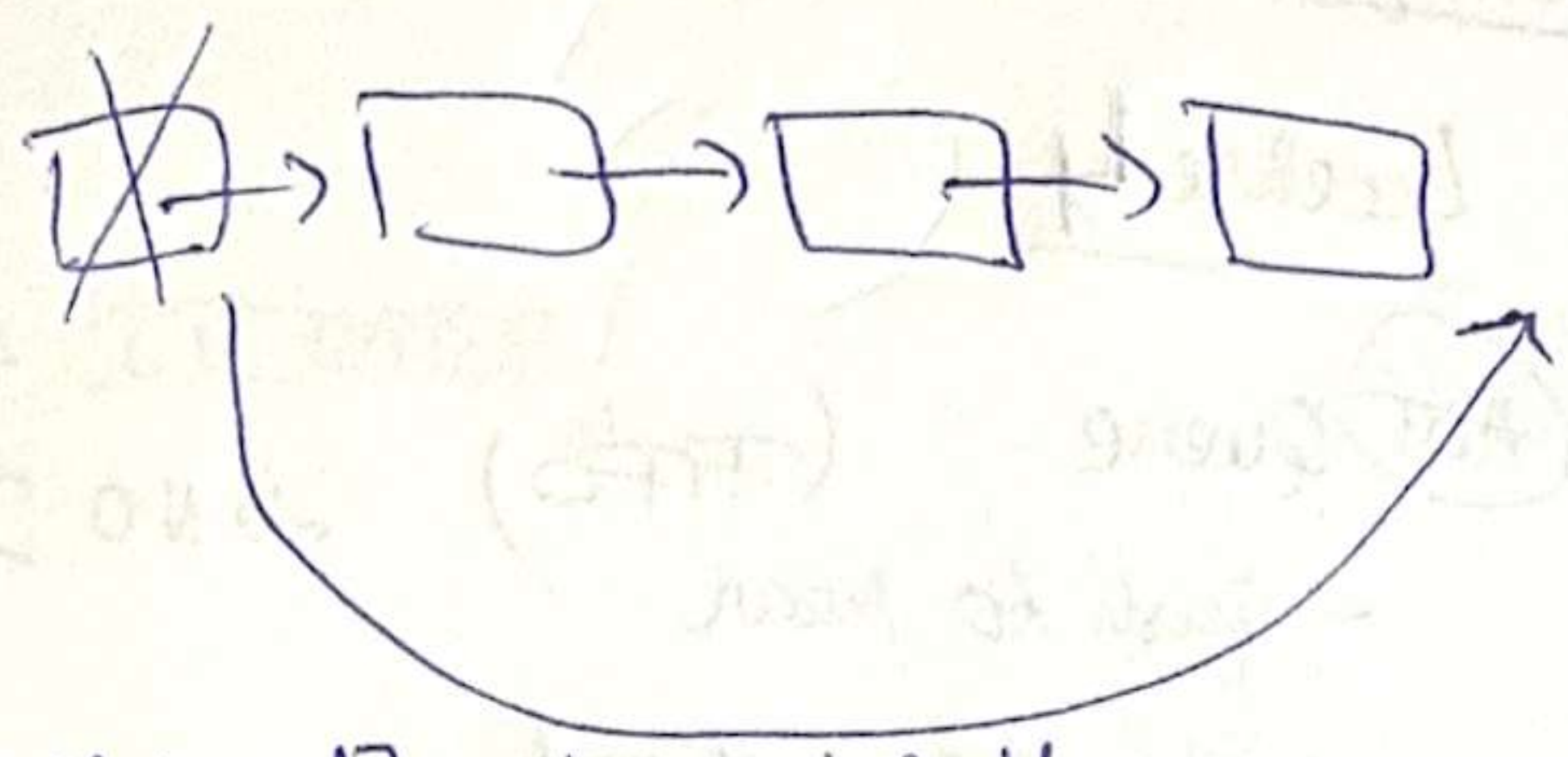
[[currentNode].next].prev ← newNode

[currentNode].next ← newNode



ROTATE SLL

- empty list
- single node list



Subalg rotate(sll) is:

if NOT (sll.head = nil OR [sll.head].next = nil) then

first ← [sll.head].next

current ← sll.head

while [current].next ≠ nil ex

current ← [current].next

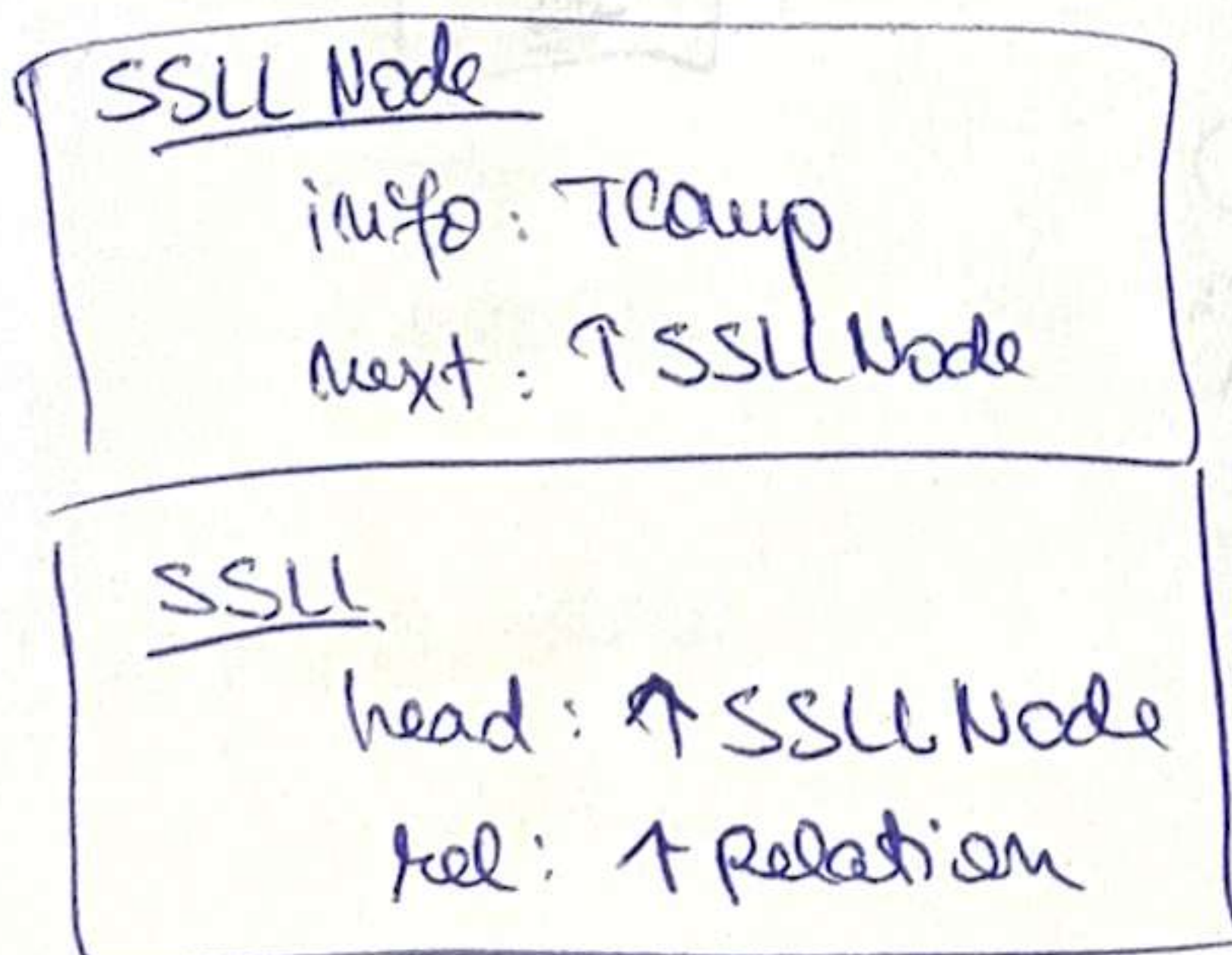
[current].next ← first

[first].next ← nil

Make sure does not point back to new head.

SortedList

I. Sorted SLL (SSLL)



Subalg insert(ssll, elem) is:

newNode ← allocate()

[newNode].info ← elem

[newNode].next ← nil

if ssll.head = nil then

ssll.head ← newNode

else if ssll.rel(elem, [ssll.head].info)

[newNode].next ← ssll.head

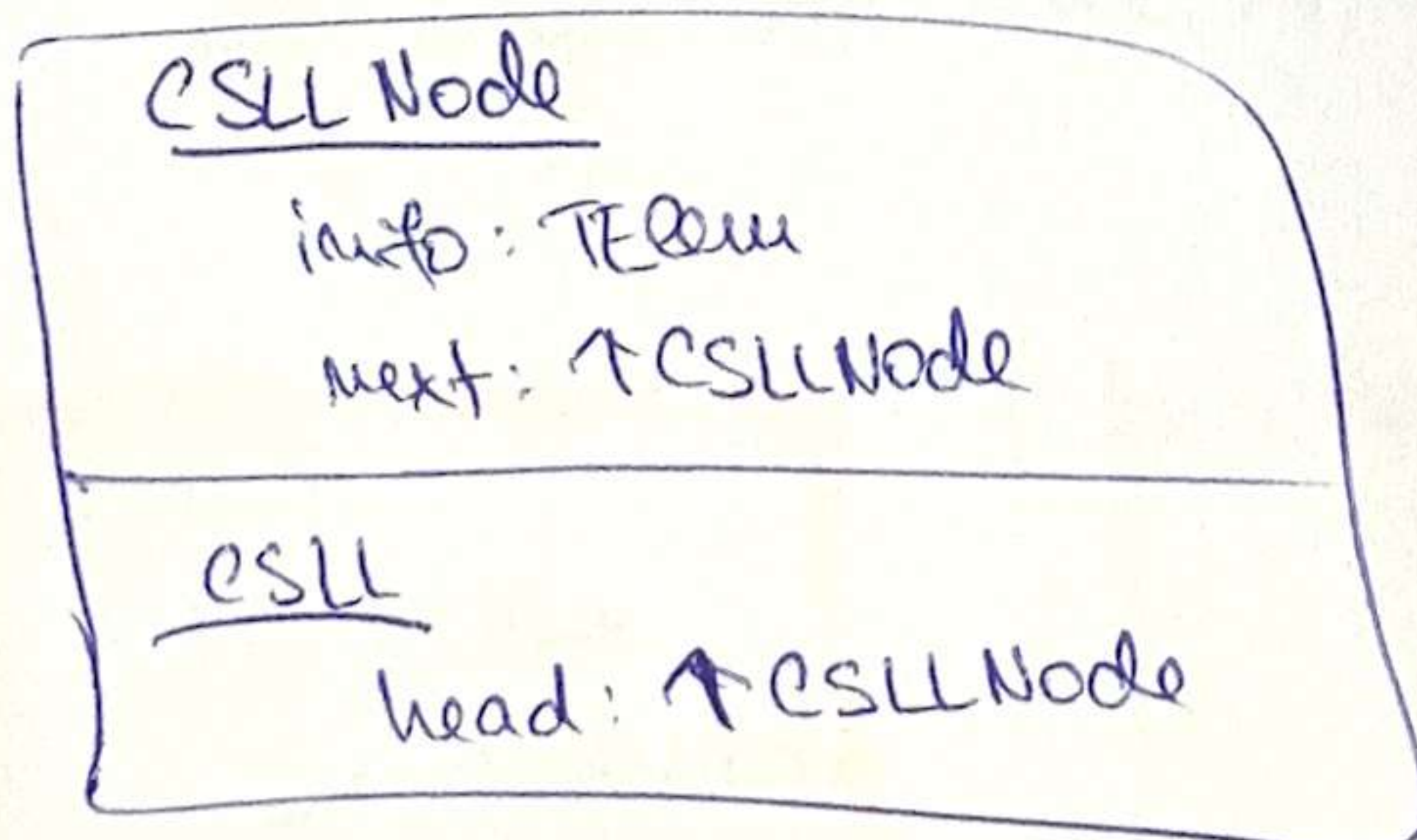
ssll.head ← newNode

else

current ← ssll.head

while --

II. circular lists



Iterator → IsValid: → next ≠ head

↳ current → next = head (invalid)

If I want to print → call [element] one more time ~

Flag

Lecture 4

1) ADT Queue (FIFO) $\rightarrow$ NO ITERATOR

- push to rear

- pop from front.

$\rightarrow$ DS: - static array
+ isFull

• init

• push

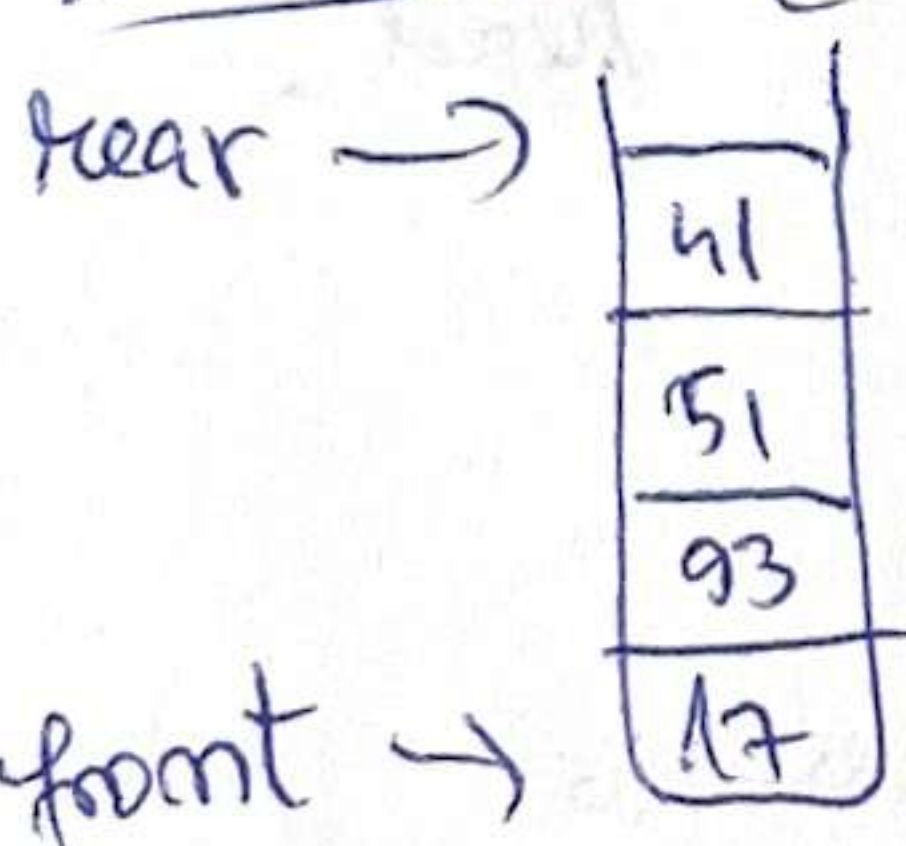
• pop $\rightarrow$ remove + return

• top $\rightarrow$ return

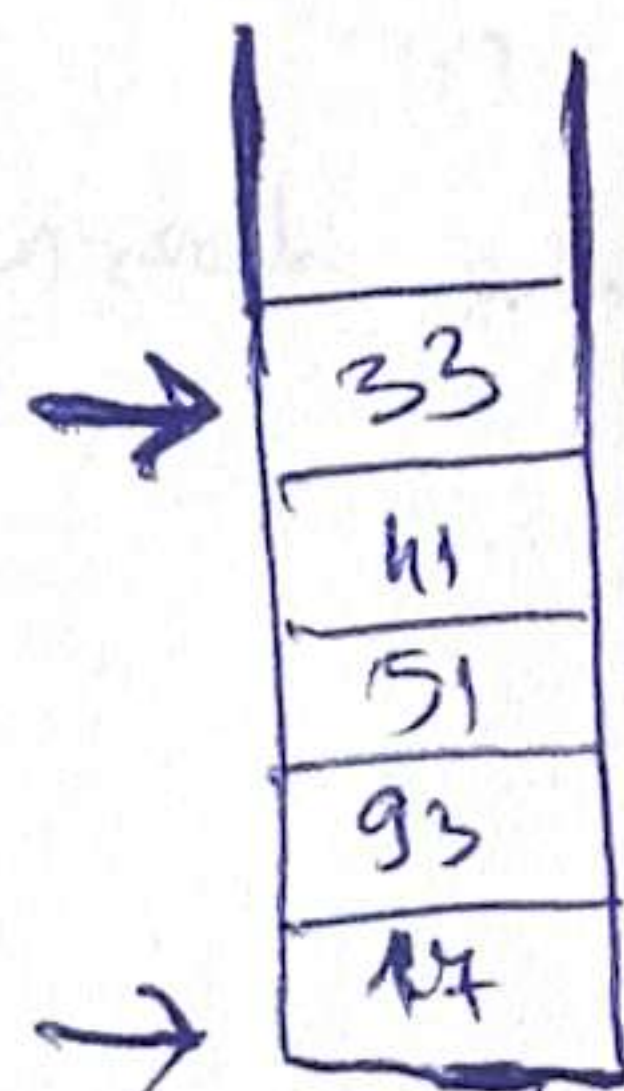
• isEmpty

• destroy

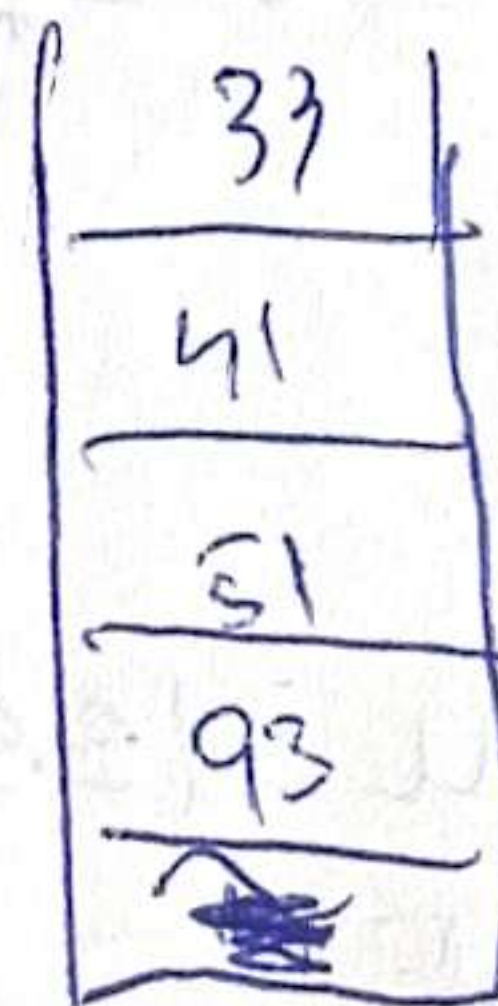
I. static array



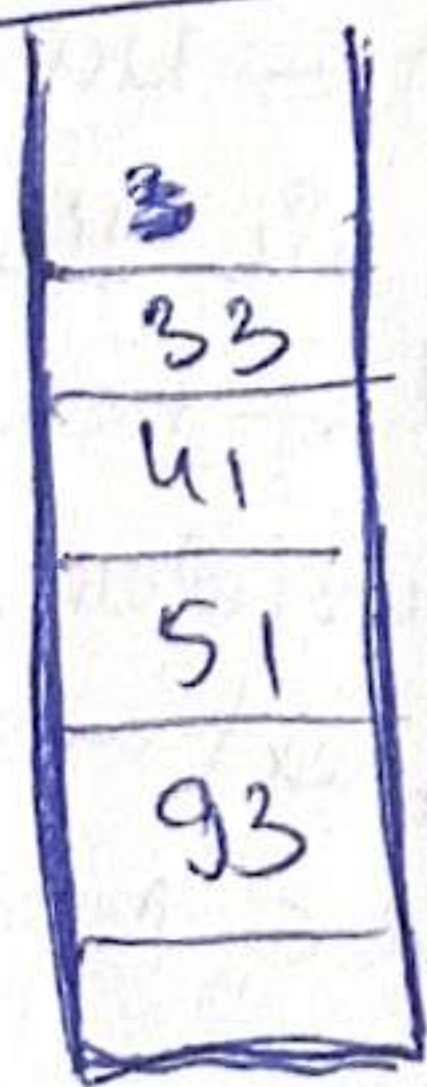
=)



=)



II. circular array



2) ADT Priority Queue (HPF)

- init
- destroy
- push
- pop
- top
- isEmpty

→ NO ITERATOR!

3) ADT Deque → insert/del from both ends.4) ADT List → POSITIONS

- init
- first
- last
- valid
- next
- previous
- getElem
- position
- setElem

→ Indexed list

→ Iterator list

5) D.A ⊕ cl. time access → add/remove at end⊖ linear ~~time~~ time → add/remove at beginning

6) LL

SUNode

info: TElem

next: TSUNode

SL

head: TSUNode

function search (sl, elem) is:

current ← sl.head

while current ≠ nil and

[current].info ≠ elem execute

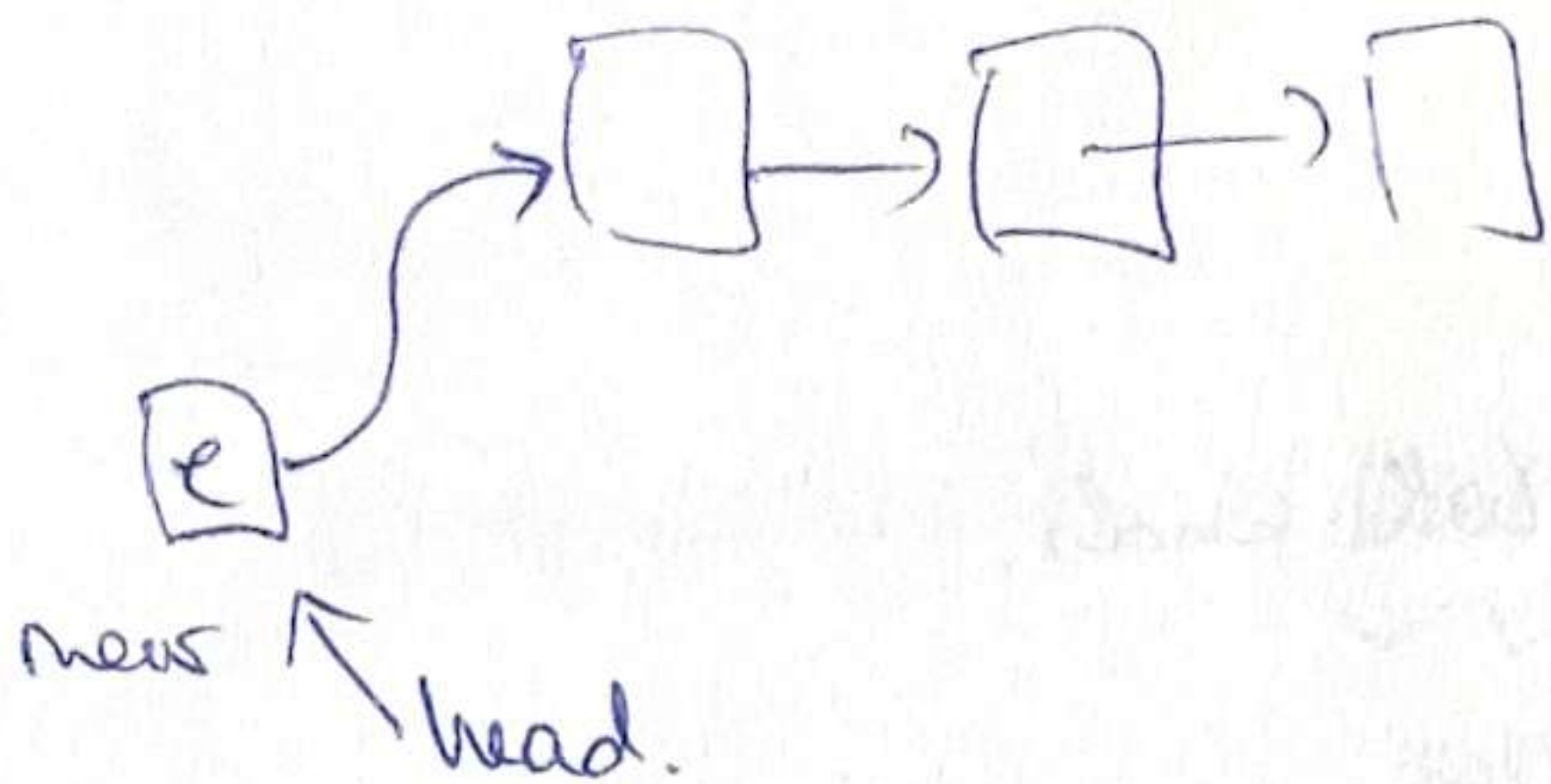
current ← [current].next

search ← current

Node not recommended
⇒ instead, pos / flag.

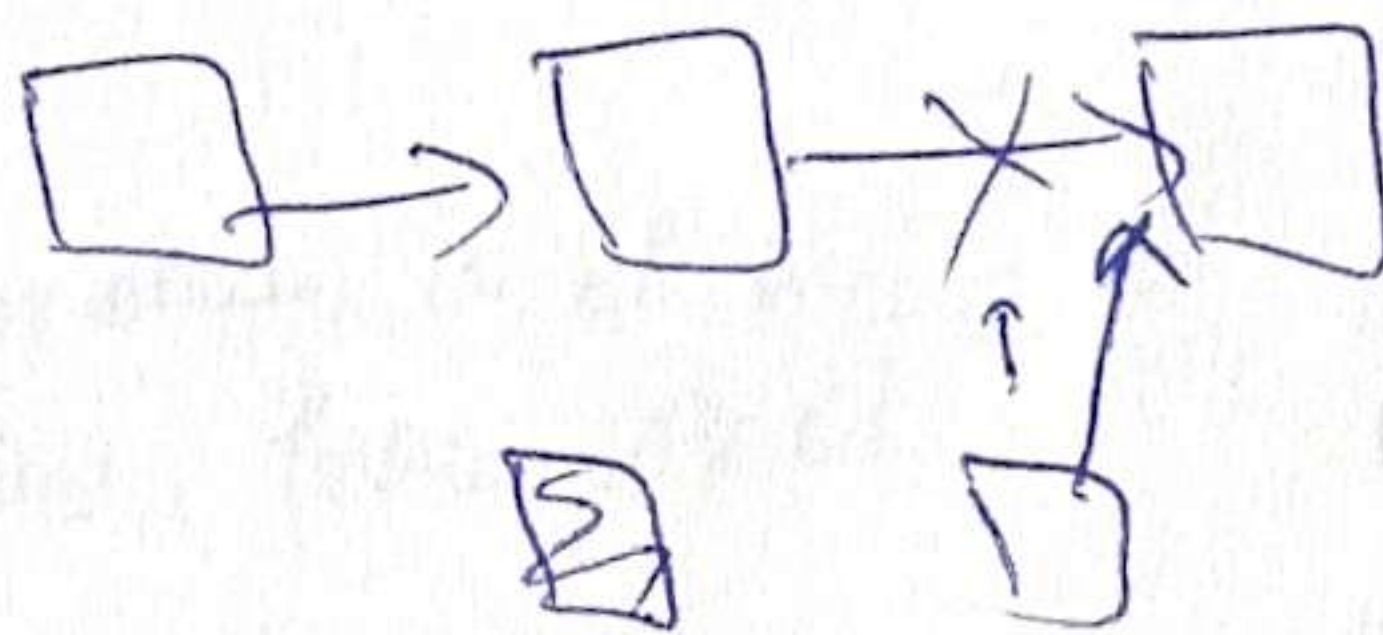
insert first
(SLL)

newNode ← allocate()
[newNode].info ← elem
[newNode].next ← self.head
self.head ← newNode



insert after

newNode ← allocate()
[newNode].info ← elem
[newNode].next ← [currentNode].next
[currentNode].next ← newNode



insert at pos 5 → we need the 4th node. = currentNode

⇒ insert after currentNode

delete → before ⇒ 2 pointers $\begin{cases} \text{currentNode} \\ \text{prevNode} \end{cases}$

Lecture 3

1) ADT Bag → DS - DA. $\begin{cases} \text{Seq}(R_1) \\ \text{w/ freq.}(R_2) \end{cases}$

Bag: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9.

R₁: 4 | 1 | 6 | 4 | 7 | 2 | 1 | 1 | 1 | 9 | -5

• add -5

• remove 6

• no order

elems	4	1	6	7	2	9
freq	2	4	1	1	1	1

4, 1, 6, 7, 2, 1, 1, 1, 9

add-5...

Matrix

Sparse matrix R1 (line, col, val)
sort

Lect 2

- Arrays
- ⊕ access any elem. in O(1) time
 - ⊖ know elems. from start.

Print container

subalg print(c) is
 iterator(c, it)
 while valid(it) execute
 elem ← getcurrent(it)
 print elem
 next(it)

Print w indexes

for i ← 1, size(da) ex
 elem ← getElem(da, i)
 print elem

Iterator DA

da: DynamicArray
 current: Int ← 0

(it.current > it.da.mrElem ⇒ throw)